



Units & Towers & BuyMenu & InfoMenu

В данном демо проекте можно было обойтись сильно примитивной системой: сразу строить башню с находящимися на ней юнитами. И при прокачке башни увеличивать дальность, урон и количество юнитов сразу. Обычно в TowerDefence играх башен много и они достаточно разнообразны, чтобы оставить систему в таком примитивном состоянии. Тогда упор можно сделать в другую сторону развития башен.

В моём случае (ограничение ассетов юнитов и башен) мне показалось, что можно сделать систему интереснее, поэкспериментировать и немного увеличить возможность прогрессии.

Графические ассеты имеют 7 этапов развития башен. Я решил, что игрок сможет строить в каждой зоне строительства башню и прокачивать её поэтапно. Это будет отдельно увеличивать дальность поражения находящихся на этой башне, а также, в определенные этапы, повышать количество юнитов на башне. Например на 3ем и 6ом уровне она будет получать +1 к размещению юнита. Однако, и юнитов тоже нужно будет отдельно нанимать.

Первый юнит будет появляться сразу со строительством башни. Однако двух последующих (менее или более, в зависимости от башни) придётся нанимать самостоятельно.

Ко всему прочему, каждого из юнитов можно прокачать отдельно, повысив его скорость атаки и урон (подразумевается наличие 1го, 2го, 3го уровней).

Возможно данная система излишне усложнена, но это просто небольшой экспериментальный проект, где я обучаюсь созданию игр и ведению документации, а посему такой эксперимент очень даже имеет место быть.

В дальнейшем, эта система позволит обыграть себя ещё интереснее - через систему артефактов, которые игрок будет

получать в конце этажа.

Units Type (Variant).

i Типы юнитов: (включая тех, что ещё ожидаются).

Юнит	Описание
Стандартный лучник	Обычный физический урон.
Лучник ледяной	Урон чуть меньше, но замедляет.
Лучник огненный	Поджигает врага, наносит переодический урон.
Лучник ядовитый	Ослабленный постепенный урон + лужа яда, наносящая урон вошедшим, появляется после смерти юнита.
Маг	Обычный силовой урон по площади.
Маг ледяной	Пониженный урон, но замедляет.
Маг огненный	Поджог. Периодический урон.
Маг желтый	Электрический урон.
Катапульта	Редкий урон по площади.
Рыцари	Пешие войны. Задерживают + атакуют.
Рыцари +2	Пешие войны с повышенными характеристиками.
Рыцари +3	Пешие войны с повышенными характеристиками.

i Характеристики юнитов(могут изменится в ходе разработки):

Юнит	Урон	Скорость атаки
Лучник +1	30	4.0
Лучник +2	55	3.0
Лучник +3	85	2.2
Маг +1		
Маг +2		
Маг +3		
Ледяной маг +1		
Ледяной маг +2		
Ледяной маг +3		
Катапульта +1		
Катапульта +2		
Катапульта +3		
Рыцарь +1		
Рыцарь +2		
Рыцарь +3		

☐ Точка, от которой стоит отталкиваться при построении радиусов атаки довольно абстрактная. Я создал на уровне стандартный Circle, и через scale подобрал примерный старт для башен. Проверил градацию в зависимости от уровня и понял, что конкретно меня устраивает.

Towers Stats.

Башня	Реальный скейл	Радиус атаки
Лучников +1	3	30
Лучников +2	3.4	34

Лучников +3	3.8	38
Лучников +4	4.2	42
Лучников +5	4.6	46
Лучников +6	5	50
Лучников +7	5.4	54

TowerDefinition | UnitDefinition.

⚠ При дальнейшей работе и создании полноценной системы пришлось работать сразу над множеством скриптов, держать в голове зависимости и функциональность. Полноценно и понятно объяснить, разделяя скрипты по частям, будет сложно, но и набрасывать всё в одну повозку тоже не лучшая идея. Возможно блоки ниже будут расположены не в лучшем порядке для понимания...

ℹ Нам нужна универсальная система хранения данных по типам башен и юнитов. Система должна иметь всё необходимое, включая внутри себя массив градации уровней, каждый объект из которых будет таким же хранилищем, но уже для определенного уровня.

```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Towers/Tower Definition")]
4 public class TowerDefinition : ScriptableObject
5 {
6     public string id;
7     public Sprite icon;
8
9     public TowerLevelData[] levels;
10
11     public UnitDefinition unitType;
12 }

```

```

1 using UnityEngine;

```

```

2
3 [System.Serializable]
4 public class TowerLevelData
5 {
6     public int level;
7
8     public float radiusScale;
9     public float attackRadiusText => radiusScale * 100;
10
11     public int unitCapacity;
12
13     public int cost;
14
15     public GameObject towerVisual;
16 }

```

Башня содержит айди в виде своего названия, иконку, поддерживаемый тип юнитов, и массив данных под каждый уровень. Каждый уровень содержит внутри индекс, радиус скейл (для вычислений), отображаемый радиус в меню покупки и информации, лимит юнитов, стоимость и спрайт. Изначально планировалось держать спрайт + анимацию, но позже появились проблемы с созданием юнитов. Их нужно было располагать в определенном пространстве, а для некоторых башен необходимы были ещё и маски (чтобы спрайт юнита не выходил за пространство окна башни). Поэтому было решено объединить анимацию, маски, спрайты, позиции спавна юнитов в один объект. Позже это будет префаб конкретного уровня башни.

```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Units/Unit Definition")]
4 public class UnitDefinition : ScriptableObject
5 {
6     public string id;
7     public UnitLevelData[] unitLevelData;
8 }

```

```

1 using UnityEngine;
2
3 [System.Serializable]
4 public class UnitLevelData
5 {
6     public Sprite icon;
7     public int level;
8     public int damage;
9     public float attackCooldown;
10    public int cost;
11    public GameObject unitPrefab;
12 }

```

Юнит хранит в себе айди-название, и хранилище данных под каждый уровень. Каждый уровень содержит иконку, индекс уровня, урон, задержку атаки, стоимость и префаб. Изначально в первом уровне данных было больше, к примеру иконка изначально планировалась в одном виде, но позже каждый уровень получил свою.

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 [CreateAssetMenu(menuName="Towers/Tower Catalog")]
5 public class TowerCatalog : ScriptableObject
6 {
7     public List<TowerDefinition> towers;
8 }

```

К прошлым хранилищам добавляется ещё одно - каталог всех башен. В итоге у нас получается три хранилища в виде ScriptableObject.

Plate buildings.

i И так, у нас есть некий префаб плитки, со специальным спрайтом, обозначающим площадку постройки. Скрипт Plate - основной скрипт, можно сказать контролер, который позволяет другим скриптам и игроку работать со всей остальной частью через него. Он отлавливает нажатие\наведение на плитку и имеет две фазы.

В первой префаб представляет площадку для постройки, при наведении он отображает случайный призрачный (голограммный) спрайт башни из хранилища таких спрайтов. При нажатии открывается меню покупки башни.



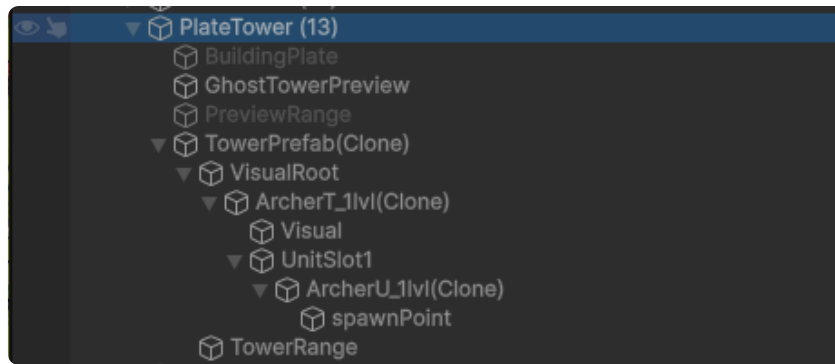
Plates.



Preview Tower.



BuyMenu.



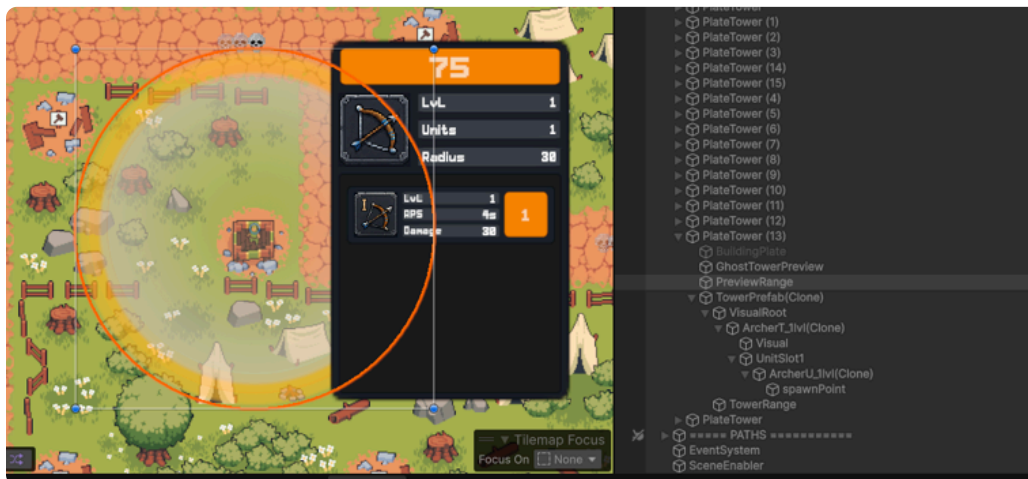
Hierarchy.

Забегая наперёд, иерархия объектов сущности башни:

Plate скрипт, спрайт плитки постройки, превью башни, превью радиус (оранжевого цвета, указывает на будущий радиус башни, в случае покупки - 1ур, в случае улучшения - следующего).

Далее идёт префаб башни, содержащий скрипт Tower, при этом он создаётся лишь раз, в отличие от того же VisualRoot. Он же также содержит синий радиус - реальный радиус, производящий вычисления входящих и выходящих врагов - TowerRange скрипт.

VisualRoot является рут объектом для хранения тех самых префабов каждого уровня башни. ArcherT_1lvl - как раз является префабом и содержит скрипт TowerVisual, вычисляет куда будут поставлены юниты, а также инициализирует юнитов, передавая им нужные зависимости. После идёт префаб юнита и точка спавна для снарядов.



Preview Range.

```

1 using DG.Tweening;
2 using System.Collections;
3 using Unity.VisualScripting;
4 using UnityEngine;
5 using UnityEngine.EventSystems;
6
7 public class Plate : MonoBehaviour,
8     IPointerEnterHandler,
9     IPointerExitHandler,

```

```

10         IPointerClickHandler
11     {
12         public Tower CurrentTower { get; private set; }
13         public bool IsEmpty => CurrentTower == null;
14
15         public CircleCollider2D radius;
16         public GameObject plateBuildings;
17
18         public SpriteRenderer previewTower;
19         public float delayShowGhostTower = 0.2f;
20
21         public float hoverScaleMultiply = 1.08f;
22         public float scaleDuration = 0.15f;
23
24         private float scale;
25         private float hoverScale;
26
27         private WaitForSeconds waitSpawnGhostTower;
28         private Coroutine WaitSpawnGhostTowerRoutine;
29
30         public GameObject towerPrefab;
31
32         public Transform previewRange;
33
34         private bool rangeIsActive;
35
36         private bool hoverRangeVisible;
37         private bool selectedRangeVisible;
38
39         private void ValidateComponents()
40         {
41             Debug.Assert(previewRange != null);
42             Debug.Assert(previewTower != null);
43         }
44
45         /*
46          * О открыл для себя новую фишку - Debug.Assert и валидация данных
47          * через неё. Когда мы избегаем в коде частых проверок на null, где
48          * объект обязан быть для функционирования, а не просто есть шанс на
49          * отсутствие. Дабы избежать случайно недобавленного компонента -
50          * используем проверку обязательных компонентов через Assert.
51          */
52
53         private void Start()
54         {
55             ValidateComponents();
56
57             previewTower.enabled = false;
58
59             waitSpawnGhostTower = new
60             WaitForSeconds(delayShowGhostTower);
61             scale = plateBuildings.transform.localScale.x *
62             hoverScaleMultiply;
63             hoverScale = scale * hoverScaleMultiply;
64
65             HidePreviewRange();
66         }
67
68         public void OnPointerEnter(PointerEventData eventData)
69         {
70             if (!IsEmpty)
71             {
72                 hoverRangeVisible = true;
73                 UpdateTowerRange();
74
75                 CurrentTower.HoverEnter();
76             }
77         }
78     }
79 }

```

```

71         else
72         {
73             WaitSpawnGhostTowerRoutine =
StartCoroutine(WaitSpawnGhostTower());
74
75             plateBuildings.transform.DOKill();
76             plateBuildings.transform
77                 .DOScale(hoverScale, scaleDuration)
78                 .SetEase(Ease.OutQuad);
79         }
80     }
81
82     /*
83     При наведении, если объект не пустой, мы включаем отображение
реального радиуса через вызов метода у башни. В случае отсутствия
башни - мы производим небольшую анимацию плавного увеличения, а также
запускаем включение превью башни с задержкой. Сделано это, чтобы при
случайном пролёте мыши экран не мерцал голубым.
84     */
85
86     public void OnPointerExit(PointerEventData eventData)
87     {
88         if (!IsEmpty)
89         {
90             hoverRangeVisible = false;
91             UpdateTowerRange();
92
93             CurrentTower.HoverExit();
94         }
95         else
96         {
97             plateBuildings.transform.DOKill();
98             plateBuildings.transform
99                 .DOScale(scale, scaleDuration)
100                 .SetEase(Ease.InQuad);
101
102             if (WaitSpawnGhostTowerRoutine != null)
103             {
104                 StopCoroutine(WaitSpawnGhostTowerRoutine);
105                 WaitSpawnGhostTowerRoutine = null;
106             }
107
108             previewTower.enabled = false;
109         }
110     }
111
112     /*
113     Когда курсор покидает башню также есть два случая: если башня
присутствовала и мы показывали радиус атаки, то мы его скрываем; если
плитка была простой - плавно возвращаем размер через Dotween методы,
выключаем превью спрайт.
114     */
115
116     public void OnPointerClick(PointerEventData eventData)
117     {
118         if (IsEmpty)
119             TowerUIManager.Instance.ToggleBuyMenu(this);
120         else
121         {
122             TowerUIManager.Instance.ToggleInfoMenu(this);
123         }
124     }
125
126     /*
127     Для клика у нас есть те же две фазы, открываем либо меню покупки,
либо меню информации о текущей башне.
128     */

```

```

129
130     private IEnumerator WaitSpawnGhostTower()
131     {
132         yield return waitSpawnGhostTower;
133
134         previewTower.sprite =
GhostTowersManager.Instance.GetRandomGhostSprite();
135         previewTower.enabled = true;
136         WaitSpawnGhostTowerRoutine = null;
137     }
138
139     public bool Build(TowerDefinition towerType)
140     {
141         if
142         (!WalletManager.Instance.TrySpend(towerType.levels[0].cost))
143             return false;
144
145         HidePreviewRange();
146
147         var tower = Instantiate(towerPrefab, transform);
148         CurrentTower = tower.GetComponent<Tower>();
149
150         CurrentTower.SetTowerDefinition(towerType);
151
152         plateBuildings.SetActive(false);
153
154         AudioManager.Instance.PlaySFXType(SoundDefaultEnum.TowerBuy);
155
156         return true;
157     }
158
159     /*
160     При успешном Build мы создаём префаб объекта со скриптом Tower, а
161     также с голубым фактическим радиусом. Плитку строительства (спрайт)
162     мы отключаем. Не забываем проиграть звук постройки.
163     */
164
165     public void ShowPreviewRangeBuy(TowerDefinition tower)
166     {
167         previewRange.localScale = Vector3.one *
168         tower.levels[0].radiusScale;
169         rangeIsActive = true;
170         previewRange.gameObject.SetActive(true);
171     }
172
173     public void ShowPreviewRange(float radius)
174     {
175         previewRange.gameObject.SetActive(true);
176         previewRange.localScale = Vector3.one * radius;
177     }
178
179     /*
180     Когда башни нет - используется метод для показа радиуса первого
181     уровня выбранной выбшни. Когда башня уже стоит, то мы используем
182     показ радиуса с параметром (башни определенного уровня).
183     */
184
185     public void HidePreviewRange()
186     {
187         previewRange.gameObject.SetActive(false);
188     }
189
190     private void UpdateTowerRange()
191     {
192         if (IsEmpty)
193             return;
194     }

```

```

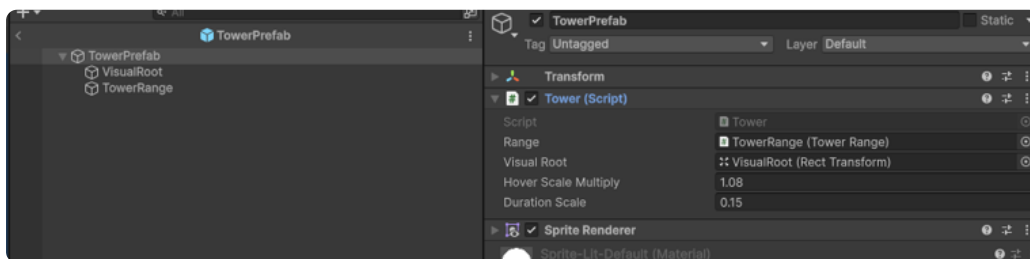
190         if (hoverRangeVisible || selectedRangeVisible)
191             CurrentTower.ShowRange();
192         else
193             CurrentTower.HideRange();
194     }
195
196     /*
197     К слову, ранее была проблема: я хранил спрайт плитки строительства
    на основном объекте plate. И когда я делал scale анимацию через
    дотвин, радиус атаки тоже увеличивался при наведении, что нарушает
    UX. Поэтому позже PlateSprite стал отдельным объектом, и скейлинг
    применяется только к нему.
198     */
199
200     public void Select()
201     {
202         selectedRangeVisible = true;
203         UpdateTowerRange();
204         rangeIsActive = true;
205     }
206
207     public void Deselect()
208     {
209         selectedRangeVisible = false;
210         UpdateTowerRange();
211     }
212 }

```

Позже код может измениться, и добавиться вызов GameModifiers. Условно, я могу решить сделать предмет, который позволит строить первые пять башен сразу второго уровня. Тогда придётся показывать радиус сразу же из башни второго уровня.

Tower.

i При покупке башни мы получаем объект Tower, скрипт Tower на нём, радиус атаки и VisualRoot.



TowerPrefab.

```

1 using DG.Tweening;
2 using System;
3 using System.Collections.Generic;
4 using UnityEngine;
5
6 public class Tower : MonoBehaviour
7 {
8     [HideInInspector] public TowerDefinition towerDef;
9     [HideInInspector] public int currentLevel;
10    [HideInInspector] public int currentUnitsCount;
11

```

```

12 [SerializeField] private TowerRange range;
13 [SerializeField] private Transform visualRoot;
14
15 private TowerVisual currentVisual;
16
17 [HideInInspector] public UnitDefinition unitDef;
18 [HideInInspector] public List<int> unitLevelsBySlot = new();
19
20 private Vector3 scale;
21 private float hoverScale;
22 [SerializeField] private float hoverScaleMultiply = 1.08f;
23 [SerializeField] private float durationScale = 0.15f;
24
25 private void Start()
26 {
27     scale = visualRoot.localScale;
28     hoverScale = scale.x * hoverScaleMultiply;
29 }
30
31 public bool TryUpgradeTower()
32 {
33     if (!WalletManager.Instance.TrySpend(
34         towerDef.levels[currentLevel].cost))
35         return false;
36
37     currentLevel++;
38
39     int capacity = towerDef.levels[currentLevel -
40 1].unitCapacity;
41
42     while(unitLevelsBySlot.Count < capacity)
43     {
44         unitLevelsBySlot.Add(0);
45     }
46
47     UpdateCurrentTowerRange();
48     RefreshVisual();
49     RestoreUnits();
50
51     if (currentLevel != towerDef.levels.Length)
52     {
53         AudioManager.Instance.PlaySFXType(SoundDefaultEnum.TowerUpgrade);
54     }
55     else
56     {
57         AudioManager.Instance.PlaySFXType(SoundDefaultEnum.TowerUpgradeMax);
58     }
59     return true;
60 }
61
62 /*
63     Улучшение башни пока не планируется в духе +2 уровня за раз,
64     поэтому метод улучшения башни работает по принципу простого
65     инкремента, а затем обновлению всех данных используя следующий
66     контейнер уровня.
67
68     Мы также храним массив доступных слотов под юнитов
69     unitLevelsBySlot. Он работает очень просто: мы храним индексы уровней
70     0, 1, 2, 3, где 0 - доступный слот для покупки. Тем самым напрямую
71     юнитов мы здесь не храним, а просто знаем, что юнит под индексом 0 в
72     данный момент например 3 уровня. [3, 3, 0].
73 */
74
75 public void SetTowerDefinition(TowerDefinition towerDef)
76 {
77     this.towerDef = towerDef;

```

```

69         unitDef = towerDef.unitType;
70
71         currentLevel = 1;                                /*
GameModifiers.Instance.StartTowerLvl;
72         unitLevelsBySlot.Clear();
73
74         int capacity = towerDef.levels[0].unitCapacity;
75
76         for (int i=0; i < capacity; i++)
77         {
78             unitLevelsBySlot.Add(0);
79         }
80
81         UpdateCurrentTowerRange();
82         RefreshVisual();
83         GiveStartUnit();
84         RestoreUnits();
85         HideRange();
86     }
87
88     /*
89     SetTowerDefinition - вызывается при покупке башни, чтобы
установить для плитки её тип. На старте также присваиваем первый
уровень, а также я сделал заготовку под модификатор.
90     */
91
92     private void GiveStartUnit()
93     {
94         unitLevelsBySlot[0] = 1;
95         currentUnitsCount++;
96     }
97
98     public bool TryUnitBuy(int index)
99     {
100         if (unitLevelsBySlot[index] != 0)
101             return false;
102         if (index >= towerDef.levels[currentLevel-1].unitCapacity)
103             return false;
104
105         if
(!WalletManager.Instance.TrySpend(unitDef.unitLevelData[0].cost))
106             return false;
107
108         unitLevelsBySlot[index] = 1;
109
110         currentUnitsCount++;
111         currentVisual.AddUnit(
112             index,
113             unitDef.unitLevelData[0].unitPrefab,
114             unitDef.unitLevelData[0]
115         );
116
117         AudioManager.Instance.PlaySFXType(SoundDefaultEnum.UnitBuy);
118
119         return true;
120     }
121
122     /*
123     Покупка юнита происходит с получением индекса. Для безопасности мы
всё же проверяем, доступен ли соответствующий индекс для покупки, хотя
в панели информации башни, где приобретается юнит уже есть все
необходимые проверки. Покупка юнита в данном коде всегда
подразумевает покупку юнита первого уровня.
124     Инициализируем внутри TowerVisual юнита и передаём нужные данные.
125     */
126
127     /*

```

```

128     Методы покупки\улучшения башен\юнитов имеют возвращаемое значение
        bool. Их можно было оставить простыми void, и тогда проверку на
        возможность постройки можно было оставить в BuyMenu\InfoMenu, однако
        строительству\улучшению может помешать другой фактор и об этом будет
        знать только сам скрипт Tower, поэтому лучше именно возвращать
        информацию о результате. Это также важно, чтобы обновлять UI панелей
        покупки и информации.
129 */
130
131     public bool TryUnitUpgrade(int index)
132     {
133         if (unitLevelsBySlot[index] == 0)
134             return false;
135         if (unitLevelsBySlot[index] >= unitDef.unitLevelData.Length)
136             return false;
137
138         int nextLevel = unitLevelsBySlot[index];
139         if
140 (!WalletManager.Instance.TrySpend(unitDef.unitLevelData[nextLevel].co
141 st))
142             return false;
143
144         unitLevelsBySlot[index]++;
145
146         var levelData = unitDef.unitLevelData[unitLevelsBySlot[index]
147 - 1];
148
149         currentVisual.UpgradeUnit(
150             index,
151             unitDef.unitLevelData[unitLevelsBySlot[index] -
152 1].unitPrefab,
153             levelData
154         );
155
156         if (unitLevelsBySlot[index] == unitDef.unitLevelData.Length)
157         {
158             AudioManager.Instance.PlaySFXType(SoundDefaultEnum.UnitMaxUpgrade);
159         }
160         else
161         {
162             AudioManager.Instance.PlaySFXType(SoundDefaultEnum.UnitUpgrade);
163         }
164
165         return true;
166     }
167
168     /*
169     Улучшение юнита происходит также через инкремент нужного слота.
170     Далее в поле присваивается хранилище под текущий уровень, а после
171     данные передаются в currentVisual.
172     */
173
174     public void ShowRange()
175     {
176         range.Show();
177     }
178
179     public void HideRange()
180     {
181         range.Hide();
182     }
183
184     public void UpdateCurrentTowerRange()
185     {
186         range.SetRadius(towerDef.levels[currentLevel -
187 1].radiusScale);

```

```

180     }
181     public void SetRangeScale(float radius)
182     {
183         range.SetRadius(radius);
184     }
185
186
187     public void HoverEnter()
188     {
189         visualRoot.transform.DOKill();
190         visualRoot.transform
191             .DOScale(hoverScale, durationScale)
192             .SetEase(Ease.OutQuad);
193     }
194     public void HoverExit()
195     {
196         visualRoot.transform.DOKill();
197         visualRoot.transform
198             .DOScale(scale, durationScale)
199             .SetEase(Ease.OutBack);
200     }
201
202     private void RefreshVisual()
203     {
204         if (currentVisual != null)
205         {
206             Destroy(currentVisual.gameObject);
207         }
208
209         currentVisual = Instantiate(
210             towerDef.levels[currentLevel - 1].towerVisual,
211             visualRoot)
212             .GetComponent<TowerVisual>();
213
214         currentVisual.Initialize(
215             this,
216             range);
217     }
218
219
220     /*
221     Для визуальной части можно использовать пул, перейти на него будет
222     не сложно, но пока было решено оставить Destroy/Instantiate. При
223     обновлении визуальной части, то есть при покупке или улучшении - мы
224     пересоздаём объект, включая юнитов внутри него.
225     */
226
227     public void RestoreUnits()
228     {
229         for (int i = 0; i < unitLevelsBySlot.Count; i++)
230         {
231             if (unitLevelsBySlot[i] == 0)
232                 continue;
233
234             UnitLevelData levelData =
235                 unitDef.unitLevelData[unitLevelsBySlot[i] - 1];
236
237             currentVisual.AddUnit(
238                 i,
239                 unitDef.unitLevelData[unitLevelsBySlot[i] -
240 1].unitPrefab,
241                 levelData
242             );
243         }
244     }

```

```

243  /*
244     Заполнение юнитов работает по принципу переинициализации всех
        юнитов разом. Это нужно как раз для того, чтобы из массива в виде [3,
        2, 0] создать полноценные сущности.
245  */

```

TowerRange.

i Радиус атаки в виде голубого круга. Вариантов реализовать его было много, мой же немного костыльный, через апдейт, без событийной системы, которая в общем-то могла быть излишней. Ибо тогда пришлось бы подписывать входящих врагов для каждого юнита, потом отписывать и прочее.

```

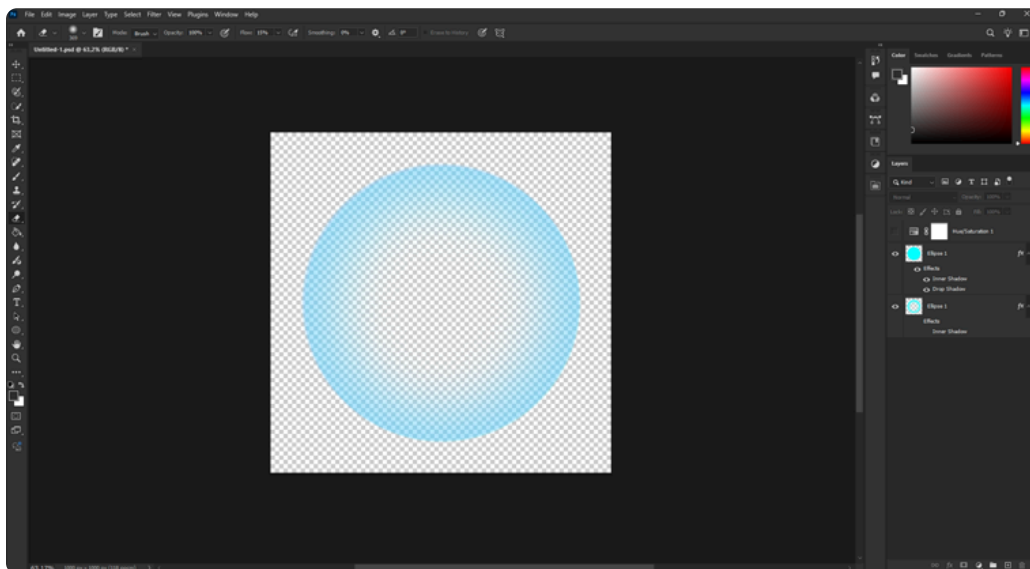
1  using NUnit.Framework;
2  using System.Collections.Generic;
3  using Unity.VisualScripting;
4  using UnityEngine;
5
6  public class TowerRange : MonoBehaviour
7  {
8      private SpriteRenderer spriteRend;
9      private Color defaultColor;
10     private Color zeroColor = new Color(0, 0, 0, 0);
11
12     private readonly List<IDamageable> targets = new();
13
14     private void Awake()
15     {
16         spriteRend = GetComponent<SpriteRenderer>();
17         defaultColor = spriteRend.color;
18     }
19
20     private void Start()
21     {
22         Debug.Assert(spriteRend != null);
23         Hide();
24     }
25
26     public void Show()
27     {
28         spriteRend.color = defaultColor;
29     }
30
31     public void Hide()
32     {
33         spriteRend.color = zeroColor;
34     }
35
36     public void SetRadius(float radius)
37     {
38         transform.localScale = Vector3.one * radius;
39     }
40
41     private void OnTriggerEnter2D(Collider2D other)
42     {
43         if (other.TryGetComponent<IDamageable>(out var target))
44         {
45             if (!targets.Contains(target))
46                 targets.Add(target);

```

```

47     }
48 }
49
50 private void OnTriggerExit2D(Collider2D other)
51 {
52     if (other.TryGetComponent<IDamageable>(out var target))
53     {
54         targets.Remove(target);
55     }
56 }
57
58 /*
59  *Храним лист вошедших в зону врагов и выкидываем вышедших.
60 */
61
62 public IDamageable GetTarget()
63 {
64     for (int i = targets.Count - 1; i >= 0; i--)
65     {
66         IDamageable target = targets[i];
67
68         if (target == null || target.IsDead)
69         {
70             targets.RemoveAt(i);
71             continue;
72         }
73         return target;
74     }
75     return null;
76 }
77 }
78
79 /*
80  *Возвращает IDamageable интерфейс врага, способный получить урон.
81  *Каждый запрос перед возвращением совершает проверку - остался ли враг
82  *живым и не является ли объект пустым.
83 */

```



Создание спрайта для радиуса атаки.

TowerVisual.

```

1 using System.Runtime.CompilerServices;
2 using UnityEngine;

```

```

3
4 public class TowerVisual : MonoBehaviour
5 {
6     [SerializeField] private Animator animator;
7     [SerializeField] private Transform[] unitSlots;
8
9     [SerializeField] private bool useMask;
10
11     private Tower tower;
12     private TowerRange range;
13
14     private Unit[] units;
15
16     public void Initialize(Tower owner, TowerRange range)
17     {
18         tower = owner;
19         this.range = range;
20     }
21
22     private void Awake()
23     {
24         units = new Unit[unitSlots.Length];
25
26         if (animator == null)
27             animator = GetComponentInChildren<Animator>();
28     }
29
30     private void Start()
31     {
32
33         UpdateSpeedAnimator(SpeedGameManager.Instance.SpeedMultiplier);
34     }
35
36     public Transform GetUnitSlot(int index)
37     {
38         return unitSlots[index];
39     }
40
41     private void OnEnable()
42     {
43         SpeedGameManager.Instance.OnSpeedGameChangedMultiplie +=
44         UpdateSpeedAnimator;
45     }
46
47     private void OnDisable()
48     {
49         SpeedGameManager.Instance.OnSpeedGameChangedMultiplie -=
50         UpdateSpeedAnimator;
51     }
52
53     private void UpdateSpeedAnimator(float speed)
54     {
55         animator.speed = speed;
56     }
57
58     public void AddUnit(int indexSlot, GameObject unitPrefab,
59     UnitLevelData data)
60     {
61         Unit unit = Instantiate(
62             unitPrefab,
63             unitSlots[indexSlot])
64             .GetComponent<Unit>();
65
66         unit.Initialize(data, tower, range);
67
68         if (useMask)

```

```

65 unit.SetMaskInteraction(SpriteMaskInteraction.VisibleInsideMask);
66
67     unit.transform.SetLocalPositionAndRotation(Vector3.zero,
Quaternion.identity);
68     units[indexSlot] = unit;
69 }
70
71 /*
72     При добавлении юнита мы создаём префаб юнита в нужной из массива
позиции. Сразу же инициализируем его, передав нужные данные.
73     Для башни лучников маски используются только на двух уровнях, и у
маски используется внутренняя часть. Поэтому, если флаг useMask -
true, то выставляем в юнита необходимую реакцию на маску.
74 */
75
76     public void RemoveUnit(int indexSlot)
77     {
78         if (units[indexSlot] == null)
79             return;
80
81         Destroy(units[indexSlot].gameObject);
82         units[indexSlot] = null;
83     }
84
85     public void UpgradeUnit(int indexSlot, GameObject newPrefab,
UnitLevelData data)
86     {
87         RemoveUnit(indexSlot);
88         AddUnit(indexSlot, newPrefab, data);
89     }
90 }
91
92 /*
93     Улучшение юнита сделано через удаление по индексу и пересозданию.
Опять же, можно перейти на object pool.
94 */

```

Unit & UnitAttack & UnitAnimationController.

i Юнитов можно было сделать одним скриптом, они чуть менее требовательны к гибкости, как те же враги, но было решено разделить примерно также. Unit - скрипт контроллер, UnitAttack - логика запуска атаки и спавна атакующего объекта, UnitAnimationController - контроль анимаций и отзеркаливания спрайта в сторону цели.

```

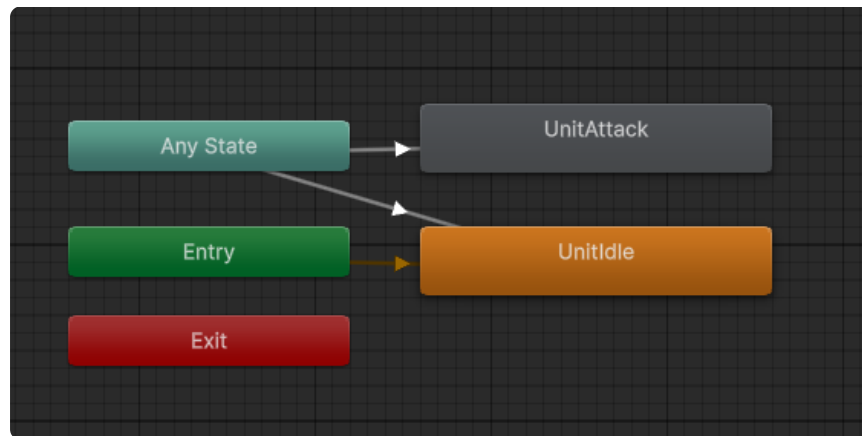
1 using UnityEngine;
2
3 public class Unit : MonoBehaviour
4 {
5     private UnitLevelData currentLevelData;
6
7     private Tower tower;
8     private TowerRange range;
9
10    private UnitAttack attack;
11    private UnitAnimationController animationController;
12

```

```

13     private IDamageable currentTarget;
14
15
16     private void Awake()
17     {
18         if (attack == null)
19             attack = GetComponent<UnitAttack>();
20         if (animationController == null)
21             animationController =
22             GetComponent<UnitAnimationController>();
23
24         Debug.Assert(attack != null);
25         Debug.Assert(animationController != null);
26     }
27
28     public void Initialize(
29         UnitLevelData data,
30         Tower owner,
31         TowerRange rangeOwner)
32     {
33         currentLevelData = data;
34         tower = owner;
35         range = rangeOwner;
36
37         attack = GetComponent<UnitAttack>();
38         animationController = GetComponent<UnitAnimationController>();
39
40         attack.Initialize(currentLevelData);
41     }
42
43     private void Update()
44     {
45         currentTarget = range.GetTarget();
46
47         if (currentTarget == null || currentTarget.IsDead)
48         {
49             animationController.PlayIdle();
50             return;
51         }
52
53         animationController.LookAt(currentTarget.Transform.position);
54
55         if (attack.TryAttack(currentTarget))
56         {
57             animationController.PlayAttack();
58         }
59     }
60
61     public void SetMaskInteraction(SpriteMaskInteraction interaction)
62     {
63         animationController.SetMaskInteraction(interaction);
64     }
65
66     /*
67     Вместо системы атаки через события, было решено использовать
68     довольно простую систему через апдейт. Мы просто каждый кадр получаем
69     доступного врага и пытаемся прокинуть его внутрь UnitAttack. Если это
70     возможно, то вызываем в добавок анимацию атаки. Если же полученный
71     таргет за долю кадра оказался мертв (застрелен другим юнитом) или
72     null, то мы просто игнорируем.
73
74     Пока я расписывал документацию у меня пришло в голову ещё одно
75     решение, это использовать в апдейте подвязку к задержке атаки, чтобы
76     уменьшить количество запросов. Впрочем, это может излишне усложнить и
77     испортить архитектуру.
78     */

```



Аниматор юнитов.

Как и у спрайтов врагов для юнитов есть разнонаправленные анимации: вверх, вниз, в сторону. Однако было решено упростить систему, да и в добавок направление вверх (спиной) немного странно будет выглядеть в закрытой башне.

Как и в случае с врагами делаем ультра простую систему анимаций - вызывать будем анимацию по имени, делаем одноименные для всех типов юнитов, создаём enum, чтобы вызывать можно было красивее.

```
1 using UnityEngine;
2
3 public enum UnitAnimation
4 {
5     UnitIdle,
6     UnitAttack
7 }
8
9 public class UnitAnimationController : MonoBehaviour
10 {
11     [SerializeField] private Animator animator;
12     [SerializeField] private SpriteRenderer spriteRenderer;
13
14     private UnitAnimation? currentAnimation;
15
16     private void Start()
17     {
18         UpdateAnimatorSpeed(SpeedGameManager.Instance.SpeedMultiplier);
19
20         PlayIdle();
21     }
22
23     private void OnEnable()
24     {
25         SpeedGameManager.Instance.OnSpeedGameChangedMultiplie +=
26         UpdateAnimatorSpeed;
27     }
28
29     private void OnDisable()
30     {
31         SpeedGameManager.Instance.OnSpeedGameChangedMultiplie -=
32         UpdateAnimatorSpeed;
33     }
34 }
```

```

31     }
32
33     private void UpdateAnimatorSpeed(float speed)
34     {
35         animator.speed = speed;
36     }
37     public void PlayIdle()
38     {
39         Play(UnitAnimation.UnitIdle);
40     }
41
42     public void PlayAttack()
43     {
44         Play(UnitAnimation.UnitAttack);
45     }
46
47     private void Play(UnitAnimation animation)
48     {
49         if (currentAnimation == animation)
50             return;
51
52         currentAnimation = animation;
53         animator.Play(animation.ToString());
54     }
55
56     public void LookAt(Vector3 targetPos)
57     {
58         spriteRenderer.flipX = targetPos.x > transform.position.x;
59     }
60
61     public void SetMaskInteraction(SpriteMaskInteraction interaction)
62     {
63         spriteRenderer.maskInteraction = interaction;
64     }
65 }

```



Archer units 1/2/3 lvl.



Archer towers 1/2/3/4/5/6/7 lvl.

```

1 using UnityEngine;
2
3 public class UnitAttack : MonoBehaviour
4 {
5     [SerializeField] private Transform projectileSpawnPoint;
6     [SerializeField] private float projectileSpeed = 4f;
7     [SerializeField] private ProjectileType typeProjectile;
8     [SerializeField] private SoundSO pushProjectileSound;
9
10    private UnitLevelData levelData;
11
12    private IDamageable target;
13    private float attackTimer;
14
15    public void Initialize(UnitLevelData levelData)
16    {
17        this.levelData = levelData;
18        attackTimer = levelData.attackCooldown;
19    }
20
21    public bool TryAttack(IDamageable target)
22    {
23        if (target == null || target.IsDead)
24            return false;
25
26        if (attackTimer > 0f)
27            return false;
28
29        this.target = target;
30        return true;
31    }
32
33    private void Update()
34    {
35        if (attackTimer > 0f)
36        {

```

```

37         attackTimer -= Time.deltaTime *
38             SpeedGameManager.Instance.SpeedMultiplier;
39     }
40 }
41
42 private void SpawnProjectile()
43 {
44     if (target == null || target.IsDead)
45         return;
46
47     var projectileGO =
48 PoolManager.Instance.Get(typeProjectile.ToString());
49
50     Projectile projectile = projectileGO.GetComponent<Projectile>
51 ();
52
53     projectile.Initialize(
54         levelData.damage,
55         projectileSpeed,
56         target,
57         projectileSpawnPoint
58     );
59
60     AudioManager.Instance.PlaySFX(pushProjectileSound);
61
62     attackTimer = levelData.attackCooldown;
63     target = null;
64 }
65 }
66
67 /*
68     Когда атака возможна и метод возвращает True - мы запускаем
69 анимацию атаки, где на последнем кадре срабатывает событие, вызывающее
70 SpawnProjectile. Мы совершаем ещё раз проверку, получаем из пула
71 объект снаряда нужного типа для данного юнита typeProjectile, получаем
72 его абстрактный класс и инициализируем. Сразу при инициализации снаряд
73 будет запущен. Включаем звук запуска снаряда, и устанавливаем таймер
74 задержки, который также зависит от скорости игры, а таргет сбрасываем.
75 */

```

```

1 using UnityEngine;
2
3 public interface IDamageable
4 {
5     void TakeDamage(float damage);
6
7     Transform Transform { get; }
8     bool IsDead { get; }
9 }
10
11 /// Для нанесения урона был создан интерфейс, который унаследовал
12 EnemyController. Мы не только наносим урон, но и храним трансформ, а
13 также состояние смерти.

```

Projectile: Arrow.

i Изначально планировалось сделать снаряды через интерфейсы, чтобы изменить нанесение стиль нанесения урона, если тот, к примеру окажется не стрелой, а огненным шаром, бьющим по радиусу. Однако логика бы повторялась, поэтому просто сделал абстрактный класс Projectile.

```

1 using UnityEngine;
2
3 public abstract class Projectile : MonoBehaviour, IPoolIdentity,
4 IPoolable
5 {
6     public ProjectileType type;
7
8     protected float damage;
9     protected float speed;
10
11     protected IDamageable target;
12
13     public SoundSO soundDamage;
14
15     public virtual void Initialize(
16         float damage,
17         float speed,
18         IDamageable target,
19         Transform spawnPosition
20     )
21     {
22         this.damage = damage;
23         this.speed = speed;
24         this.target = target;
25
26         transform.SetPositionAndRotation(
27             spawnPosition.position,
28             spawnPosition.rotation);
29     }
30
31     public string GetPoolId()
32     {
33         return type.ToString();
34     }
35
36     protected virtual void Update()
37     {
38         if (target == null || target.IsDead)
39         {
40             ReturnToPool();
41             return;
42         }
43
44         Vector3 direction =
45             target.Transform.position - transform.position;
46
47         if (direction != Vector3.zero)
48         {
49             transform.right = direction;
50         }
51
52         transform.position = Vector3.MoveTowards(
53             transform.position,
54             target.Transform.position,
55             speed *
56             Time.deltaTime *
57             SpeedGameManager.Instance.SpeedMultiplier
58         );
59
60         if (Vector3.Distance(
61             transform.position,
62             target.Transform.position) < 0.05f)
63         {
64             Hit();
65         }
66     }
67 }

```

```

66
67 /*
68     Когда мы устанавливаем таргет в инициализации, то апдейт сразу
        пропускает нас глубже и начинает двигать снаряд. Снаряд как-бы
        прилипает к врагу и обязательно достигнет его, если был выпущен. При
        движении врага снаряд также меняет своё вращение, чтобы двигаться
        передом снаряда прямо в цель (остриём стрелы).
69     При достижении цели мы вызываем Hit(). Позже этот метод
        переопределяет дочерний класс, реализующий определенный снаряд.
70 */
71
72     protected virtual void Hit()
73     {
74         AudioManager.Instance.PlaySFX(soundDamage);
75     }
76
77     protected virtual void ReturnToPool()
78     {
79         PoolManager.Instance.Return(gameObject);
80     }
81
82     public virtual void OnSpawn() { }
83     public virtual void OnDespawn()
84     {
85         damage = 0;
86         speed = 0;
87         target = null;
88     }
89 }

```

```

1  using UnityEngine;
2
3  public class Arrow : Projectile
4  {
5      protected override void Hit()
6      {
7          base.Hit();
8          target.TakeDamage(damage);
9          ReturnToPool();
10     }
11 }
12 /// Моментальное нанесение урона.

```

TowerUIManager.

i Создание всей системы башен я начинал на самом деле именно отсюда. Я всё же решил, что логичнее будет поставить эту часть в конец.

И так... Создаём менеджер, который будет выдавать нам одну из двух панелей: покупки, если башни нет, и информации, если башня есть. Более того, много времени было потрачено не столько на код или дизайн, как на то, чтобы оптимизировать панели и сделать так, чтобы при наведении на башню они появлялись в нужном месте. Учитывая, что я работаю через паттерн PersistentScene - базовую сцену, где лежат все

менеджеры и постоянный UI - это составляло небольшую трудность, но даже скорее из-за того, что я не сталкивался с подобным ранее. Решением оказалось брать координаты относительно экрана, а не относительно камер, потому что для персистент и уровневой сцены - у меня две разные камеры, которые работают в разных режимах.

Следующей проблемой стала необходимость создавать панели так, чтобы они не выходили за границы экрана (если башня например слишком низко или высоко). При решении также были добавлены ручные отступы, чтобы панель не только не выходила, но и была на некотором расстоянии от краёв экрана.





Наведение на пустую ячейку.



Меню информации башни.



Отступ от края экрана.

```

1 using DG.Tweening;
2 using UnityEngine;
3
4 public class TowerUIManager : Manager<TowerUIManager>
5 {
6     [Header("Panels")]
7     public GameObject buyPanel;
8     public GameObject infoPanel;
9
10    private BuyMenu buyMenu;
11    private InfoMenu infoMenu;
12
13    [Header("Offset")]
14    [SerializeField] private float verticalOffset = 0.6f;
15    [SerializeField] private float horizontalOffset = 1.5f;
16    [SerializeField] private float topHudPadding = 0.05f;
17
18    [Header("Utils")]
19    [SerializeField] private UIButtonSound sound;
20
21    private Vector3 verticalOffsetV3;
22
23    private RectTransform buyRect;
24    private RectTransform infoRect;
25
26    private CanvasGroup rootCG;
27    private Canvas canvas;
28
29    private Camera cam;
30
31    private Vector3 buyDefaultScale;
32    private Vector3 infoDefaultScale;
33
34    private Plate currentPlate;
35
36    protected override void Awake()
37    {

```

```

38     base.Awake();
39
40     cam = Camera.main;
41
42     buyRect = buyPanel.GetComponent<RectTransform>();
43     infoRect = infoPanel.GetComponent<RectTransform>();
44
45     buyDefaultScale = buyRect.localScale;
46     infoDefaultScale = infoRect.localScale;
47
48     rootCG = GetComponent<CanvasGroup>();
49     canvas = GetComponent<Canvas>();
50
51     verticalOffsetV3 = new(0, verticalOffset, 0);
52
53     topHudPadding *= Screen.height;
54
55     if (sound == null)
56         sound = GetComponent<UIButtonSound>();
57
58     HideInstant();
59 }
60
61 private void Start()
62 {
63     WorldCameraService.Instance.UpdateActiveCam +=
SetWorldCamera;
64     InputManager.OnBackgroundClick += HideMenuOnBackgroundClick;
65
66     buyMenu = buyPanel.GetComponent<BuyMenu>();
67     infoMenu = infoPanel.GetComponent<InfoMenu>();
68
69     buyMenu.OnTowerSelected += tower =>
70     {
71         if (tower != null && currentPlate.Build(tower))
72         {
73             HideBuyMenu();
74             sound.PlayClickSound();
75         }
76         else
77         {
78             sound.PlayErrorSound();
79         }
80     };
81
82     buyMenu.OnTowerHoverShowInfo += BuyMenuHoverTD;
83
84     infoMenu.OnHoverUpgradeTower += OnHoveredUpgradeButton;
85     infoMenu.OnExitUpgradeTower += OnExitedUpgradeButton;
86     infoMenu.ClickUpgradeButton += OnClickUpgradeButton;
87     infoMenu.OnClickCantUpgrade += sound.PlayErrorSound;
88
89     infoMenu.OnCantBuyUnit += sound.PlayErrorSound;
90     infoMenu.OnCantUpUnit += sound.PlayErrorSound;
91     infoMenu.OnHoverUnit += sound.PlayHoverSound;
92 }
93 /*
94     Вызов звуков не во всех случаях было удобно делать именно здесь,
некоторые звуки было проще вызвать рядом с остальным кодом, однако
было решено централизовать логику вызовов звуков, поэтому через
события все реакции UI доходят до сюда, и здесь же есть прямые
подписки на вызов звука, если понадобится добавить функционал, то
вместо подписки на sound, создадим другой метод с нужным
функционалом.
95 */
96 private void OnClickUpgradeButton(TowerLevelData levelData)
97 {

```

```

98         if (levelData == null)
99             currentPlate.HidePreviewRange();
100        else
101            currentPlate.ShowPreviewRange(levelData.radiusScale);
102    }
103
104    private void HideInstant()
105    {
106        if (currentPlate != null)
107        {
108            currentPlate.Deselect();
109            currentPlate = null;
110        }
111
112        buyPanel.SetActive(false);
113        infoPanel.SetActive(false);
114    }
115
116    //////////////////////////////////
117    ////////// BUY_MENU
118    //////////////////////////////////
119
120    public void ToggleBuyMenu(Plate tower)
121    {
122        if (currentPlate == tower)
123        {
124            HideBuyMenu();
125            return;
126        }
127
128        ShowBuyMenu(tower);
129    }
130
131    public void ShowBuyMenu(Plate tower)
132    {
133        HideInstant();
134        currentPlate = tower;
135
136        ShowPanel(buyPanel, buyRect, buyDefaultScale, tower);
137    }
138
139    public void HideBuyMenu()
140    {
141        HidePanel(buyPanel, buyRect, buyDefaultScale);
142        currentPlate = null;
143    }
144
145    //////////////////////////////////
146    ////////// INFO_MENU
147    //////////////////////////////////
148
149
150    public void ToggleInfoMenu(Plate tower)
151    {
152        if (currentPlate == tower)
153        {
154            HideInfoMenu();
155            return;
156        }
157
158        ShowInfoMenu(tower);
159    }
160
161    public void ShowInfoMenu(Plate tower)
162    {
163        HideInstant();
164        currentPlate = tower;

```

```

165         ShowPanel(infoPanel, infoRect, infoDefaultScale, tower);
166
167         infoMenu.SetTower(tower.CurrentTower);
168         tower.Select();
169     }
170
171     public void HideInfoMenu()
172     {
173         HidePanel(infoPanel, infoRect, infoDefaultScale);
174
175         if (currentPlate != null)
176         {
177             currentPlate.Deselect();
178             currentPlate = null;
179         }
180     }
181
182     /*
183     По сути у нас просто переключатели. Методы вызываются из Plate, а
184     закрыть или открыть решает сам TowerUIManager.
185     */
186
187     private void ShowPanel(GameObject panelGO, RectTransform panel,
188     Vector3 targetScale, Plate tower)
189     {
190         panelGO.SetActive(true);
191         panel.localScale = targetScale;
192
193         SetPanelPosition(panel, tower.transform);
194
195         rootCG.alpha = 0;
196         panel.localScale = targetScale * 0.7f;
197
198         rootCG.blocksRaycasts = true;
199         rootCG.interactable = true;
200
201         rootCG.DOFade(1f, 0.25f);
202
203         panel.DOKill();
204         panel.DOScale(targetScale * 1.05f, 0.18f)
205             .SetEase(Ease.OutBack)
206             .OnComplete(() => panel.DOScale(targetScale, 0.08f));
207     }
208
209     private void HidePanel(GameObject panelGO, RectTransform panel,
210     Vector3 targetScale)
211     {
212         rootCG.blocksRaycasts = false;
213         rootCG.interactable = false;
214
215         rootCG.DOFade(0f, 0.15f);
216
217         panel.DOKill();
218         panel.DOScale(targetScale * 0.8f, 0.15f)
219             .OnComplete(() => panelGO.SetActive(false));
220     }
221
222     /*
223     Скрываем и раскрываем панель вместе с плавной анимацией
224     вскакивания/выпрыгивания.
225     */
226
227     private void SetPanelPosition(RectTransform panel, Transform
228     tower)
229     {
230         panel.position = tower.position + verticalOffsetV3;
231     }

```

```

227     Vector3 viewportPos =
cam.WorldToViewportPoint(tower.position);
228
229     panel.position += (viewportPos.x < 0.5f)
230         ? Vector3.right * horizontalOffset
231         : Vector3.left * horizontalOffset;
232
233     Vector3[] corners = new Vector3[4];
234
235
236     panel.GetWorldCorners(corners);
237
238     float bottom = cam.WorldToScreenPoint(corners[0]).y;
239     float top = cam.WorldToScreenPoint(corners[1]).y;
240
241     float correction = 0;
242
243     if (bottom < 0)
244         correction = -bottom;
245     else if (top > Screen.height)
246         correction = (Screen.height - topHudPadding) - top;
247
248     Vector3 worldUp = cam.transform.up;
249     panel.position += worldUp * correction
250         * (Vector3.Distance(cam.transform.position,
panel.position) / cam.pixelHeight);
251 }
252
253 /*
254     Устанавливаем позицию панели ровно в позицию башни + специальный
сдвиг по высоте, потому что центром башни является пивот у низа
постройки, а не центр.
255     Переводим позицию башни в viewport координаты камеры: 0-1, 0-1,
чтобы понять, относительно центра в какой стороне башня.
256     Если башня слева от центра, то панель сдвигаем вправо, или
наоборот, если башня справа от центра. Тем самым панель всегда
стремится к центру экрана.
257
258     corners - содержит координаты каждого угла панели. Берём у bottom-
left, и top-left Y координату.
259     Если панель ушла ниже экрана, то присвоим в коррекцию
противоположное значение. Например если -50 (ниже 0), то коррекцией
будет +50 пикселей.
260     Если же панель ушла выше, то берём высоту экрана, отнимаем
координату панели, и дополнительно отнимаем отступ. Позже я добавлю
отступ снизу. Отступ присутствует, чтобы опустится ниже верхней части
интерфейса (панель с таймером, волнами, жизнью, монетами и
контроллером скорости). Например Y верхней части оказался на 1100
пикселе, хотя экран 1080, плюс панель 20 или 30, поэтому у нас
получается 1080-20-1100 = -40 (отпустить на 40).
261
262     Теперь прибавим новые координаты. Однако ещё необходимо перевести
пиксели в правильные координаты. Мы умножаем верх камеры в мировых
координатах на коррекцию и на расчёт примерного масштаба экранных
пикселей относительно мира на текущей дистанции от камеры.
263
264     Мы не проверяем выход за боковые границы, потому что наша панель
всегда стремится к центру экрана, а значит самая боковая башня будет
устанавливать панель гарантированно в границах экрана.
265 */
266
267     public void SetWorldCamera(Camera levelCamera)
268     {
269         cam = levelCamera;
270         canvas.worldCamera = levelCamera;
271     }
272

```

```

273     private void BuyMenuHoverTD(TowerDefinition tower)
274     {
275         if (tower == null)
276         {
277             sound.PlayHoverSound();
278             return;
279         }
280
281         currentPlate.ShowPreviewRangeBuy(tower);
282         sound.PlayHoverSound();
283     }
284
285     private void BuyMenuExit()
286     {
287         currentPlate.HidePreviewRange();
288     }
289
290
291     private void OnHoveredUpgradeButton(float radius)
292     {
293         currentPlate.ShowPreviewRange(radius);
294         sound.PlayHoverSound();
295     }
296
297     private void OnExitedUpgradeButton()
298     {
299         currentPlate.HidePreviewRange();
300     }
301
302     private void HideMenuOnBackgroundClick()
303     {
304         if (buyPanel.activeSelf)
305         {
306             HideBuyMenu();
307             return;
308         }
309         if (infoPanel.activeSelf)
310         {
311             HideInfoMenu();
312         }
313     }

```

BuyMenu, BuyPlateTower.

```

1  using System;
2  using System.Collections.Generic;
3  using TMPro;
4  using UnityEngine;
5  using UnityEngine.UI;
6
7  public class BuyMenu : MonoBehaviour
8  {
9      [Header("TextFields")]
10     [SerializeField] private TextMeshProUGUI unitCapacity_txt;
11     [SerializeField] private TextMeshProUGUI speedAttack_txt;
12     [SerializeField] private TextMeshProUGUI damage_txt;
13     [SerializeField] private TextMeshProUGUI cost_txt;
14
15     [Header("LibertyTower")]
16     [SerializeField] private TowerCatalog catalog;
17
18     [Header("Other")]
19     [SerializeField] private Transform rootContent;
20     [SerializeField] private GameObject prefabPlate;
21     [SerializeField] private GameObject downInfo;

```

```

22 [SerializeField] private ScrollRect scrollRect;
23 [SerializeField] private Color normalTextColor;
24 [SerializeField] private Color unknownTextColor;
25
26 private List<BuyPlateTower> plates = new();
27 private TowerDefinition selectedTower;
28
29 public event Action<TowerDefinition> OnTowerSelected;
30 public event Action<TowerDefinition> OnTowerHoverShowInfo;
31 public event Action OnTowerHideInfo;
32
33
34 public void Start()
35 {
36     CreateCatalog();
37     HideInfo();
38
39     WalletManager.Instance.OnCoinsChanged += UpdateCostText;
40 }
41
42 public void CreateCatalog()
43 {
44     if (catalog.towers.Count < 12)
45     {
46         for (int i = 0; i < 12; i++)
47         {
48             AddTowerPlate();
49         }
50     }
51     else
52     {
53         foreach (var item in catalog.towers)
54         {
55             AddTowerPlate();
56         }
57     }
58
59     for (int i = 0; i < plates.Count; i++)
60     {
61         if (i >= catalog.towers.Count || catalog.towers[i] ==
62 null)
63             plates[i].SetUnknown();
64         else
65             plates[i].SetTower(catalog.towers[i]);
66
67         plates[i].OnHover += SetTower;
68         plates[i].OnClick += BuyTower;
69         plates[i].OnExit += HideInfo;
70         plates[i].OnHoverIsUnknown += ShowUnknownInfo;
71     }
72 }
73
74 /*
75     Создаём каталог всех типов башен. Если типов башен менее 12, то
76     сначала мы создаём 12 пустых плиток. Если же башен достаточно, то
77     просто создаём столько, сколько их есть. Далее проходимся по списку,
78     если индекс созданной ячейки ссылается на индекс отсутствующей башни,
79     то мы объявляем её Unknown пустышкой. А если башня существует, то
80     вызываем SetTower. Для каждой плитки подписываемся на события.
81 */
82
83 private void AddTowerPlate()
84 {
85     var tp = Instantiate(prefabPlate, rootContent)
86         .GetComponent<BuyPlateTower>(out var plate);
87
88     plate.SetScrollRect(scrollRect);

```

```

83
84     plates.Add(plate);
85 }
86
87 /*
88  Т.к. BuyMenu живёт в виде одного экземпляра, и не пересоздаётся,
    то мы просто инстантируем плитки, и больше их не удаляем. В моей игре
    не требуется увеличения их количества.
89 */
90
91     private void SetTower(TowerDefinition tower)
92     {
93         selectedTower = tower;
94         ShowInfo();
95     }
96
97     private void HideInfo()
98     {
99         downInfo.SetActive(false);
100
101         unitCapacity_txt.text = "";
102         speedAttack_txt.text = "";
103         damage_txt.text = "";
104         cost_txt.text = "";
105
106         selectedTower = null;
107         OnTowerHideInfo?.Invoke();
108     }
109
110
111     private void ShowInfo()
112     {
113         downInfo.SetActive(true);
114
115         unitCapacity_txt.color = normalTextColor;
116         speedAttack_txt.color = normalTextColor;
117         damage_txt.color = normalTextColor;
118
119         unitCapacity_txt.text =
120         selectedTower.levels[selectedTower.levels.Length-
121         1].unitCapacity.ToString();
122         speedAttack_txt.text = $"
123         {selectedTower.unitType.unitLevelData[0].attackCooldown.ToString()}s"
124         ;
125         Debug.Log($"
126         {selectedTower.unitType.unitLevelData[0].attackCooldown.ToString()}s"
127         );
128         damage_txt.text =
129         selectedTower.unitType.unitLevelData[0].damage.ToString();
130         cost_txt.text = selectedTower.levels[0].cost.ToString();
131
132         UpdateCostText();
133
134         OnTowerHoverShowInfo?.Invoke(selectedTower);
135     }
136
137 /*
138  При наведении на иконку башни в меню покупки мы записываем её
    TowerDefinition в поле, и через ShowInfo указываем её конфигурацию.
    Для панелей код получился громоздким за счёт того, что нужно хранить
    кнопки, цвета, текста кнопок, текста статистики, цвет для различных
    случаев показа статистики и прочее.
139 */
140
141     private void ShowUnknownInfo()
142     {
143         downInfo.SetActive(true);

```

```

137
138     unitCapacity_txt.text = "XXX";
139     unitCapacity_txt.color = unknownTextColor;
140
141     speedAttack_txt.text = "XXX";
142     speedAttack_txt.color = unknownTextColor;
143
144     damage_txt.text = "Unknown";
145     damage_txt.color = unknownTextColor;
146
147     cost_txt.text = "Unknown";
148     cost_txt.color = unknownTextColor;
149
150     OnTowerHoverShowInfo?.Invoke(null);
151 }
152
153 private void UpdateCostText(int x)
154 {
155     UpdateCostText();
156 }
157 private void UpdateCostText()
158 {
159     if (selectedTower == null)
160         return;
161     if (!CanBuy())
162     {
163         cost_txt.color = Color.red;
164         return;
165     }
166
167     cost_txt.color = Color.orange;
168 }
169 private bool CanBuy()
170 {
171     return WalletManager.Instance.Coins >=
selectedTower.levels[0].cost;
172 }
173
174 private void BuyTower(TowerDefinition tower)
175 {
176     OnTowerSelected?.Invoke(tower);
177 }
178 }

```

```

1 using DG.Tweening;
2 using System;
3 using UnityEngine;
4 using UnityEngine.EventSystems;
5 using UnityEngine.UI;
6
7 /*
8     Отдельно делаем скрипт для каждой плитки башни для анимации и
9     отслеживания взаимодействий с плитками (наведение, клик), чтобы позже
10    передать это выше через события.
11 */
12
13 public class BuyPlateTower : MonoBehaviour,
14     IPointerEnterHandler,
15     IPointerExitHandler,
16     IPointerClickHandler,
17     IScrollHandler
18 {
19     public Sprite unknownTower;
20     public Image icon;
21     private bool isUnknown;
22 }

```

```

21     private TowerDefinition towerType;
22
23     private Vector3 defaultScale;
24
25     public event Action<TowerDefinition> OnHover;
26     public event Action OnExit;
27     public event Action<TowerDefinition> OnClick;
28     public event Action OnHoverIsUnknown;
29
30     private ScrollRect scrollRect;
31
32     private void Start()
33     {
34         defaultScale = icon.rectTransform.localScale;
35     }
36     public void OnHoverButton()
37     {
38         icon.rectTransform.DOKill();
39         icon.rectTransform
40             .DOScale(defaultScale * 0.95f, 0.1f)
41             .SetEase(Ease.OutQuad);
42
43         if (isUnknown)
44             OnHoverIsUnknown?.Invoke();
45         else
46             OnHover?.Invoke(towerType);
47     }
48
49     public void OnExitButton()
50     {
51         icon.rectTransform.DOKill();
52         icon.rectTransform
53             .DOScale(defaultScale, 0.1f)
54             .SetEase(Ease.OutBack);
55
56         OnExit?.Invoke();
57     }
58
59     public void OnClickButton()
60     {
61         if (isUnknown)
62             OnClick?.Invoke(null);
63         OnClick?.Invoke(towerType);
64     }
65
66     public void SetUnknown()
67     {
68         icon.sprite = unknownTower;
69         isUnknown = true;
70     }
71     public void SetTower(TowerDefinition tower)
72     {
73         this.towerType = tower;
74         icon.sprite = tower.icon;
75     }
76
77     public void OnPointerEnter(PointerEventData eventData)
78     {
79         OnHoverButton();
80     }
81
82     public void OnPointerExit(PointerEventData eventData)
83     {
84         OnExitButton();
85     }
86
87     public void OnPointerClick(PointerEventData eventData)

```

```

88     {
89         OnClickButton();
90     }
91
92     public void SetScrollRect(ScrollRect rect)
93     {
94         scrollRect = rect;
95     }
96     public void OnScroll(PointerEventData eventData)
97     {
98         scrollRect.OnScroll(eventData);
99     }
100 }

```

InfoMenu.

```

1  using DG.Tweening;
2  using System;
3  using System.Collections.Generic;
4  using System.Runtime.CompilerServices;
5  using TMPPro;
6  using UnityEngine;
7  using UnityEngine.UI;
8
9  public class InfoMenu : MonoBehaviour
10 {
11     [Header("Fields Stats")]
12     public Image iconTower;
13     public TextMeshProUGUI level_txt;
14     public TextMeshProUGUI unitsCount_txt;
15     public TextMeshProUGUI radius_txt;
16
17
18     [Header("Button")]
19     public Button upgradeButton;
20     public TextMeshProUGUI cost_txt;
21     public Color upgradeButtonColor;
22     public Color upgradeButtonTextColor;
23     public Color maxButtonColor;
24     private Image buttonImage;
25     private Color defaultButtonColor;
26     private Color defaultButtonTextColor;
27     private Vector3 defaultScaleButton;
28
29     [Header("Text Colors")]
30     public Color defaultTextColor;
31     public Color nextLevelColor;
32     public Color maxLevelColorText;
33     public Color maxButtonTextColor;
34
35     [Header("Scroll_View")]
36     public Transform contentRoot;
37     public GameObject prefabPanelUnits;
38     private List<UnitPanel> unitPanels = new();
39
40     private Tower selectedTower;
41     private TowerLevelData nextLevelData;
42     private TowerLevelData currentLevelData;
43
44     bool isHoverButton = false;
45
46     private string hexNextLevelColor;
47
48     public event Action<float> OnHoverUpgradeTower;
49     public event Action OnExitUpgradeTower;

```

```

50     public event Action<TowerLevelData> ClickUpgradeButton;
51     public event Action OnClickCantUpgrade;
52
53     public event Action OnCantBuyUnit;
54     public event Action OnCantUpUnit;
55     public event Action OnHoverUnit;
56
57     /*
58      * И снова огромное множество полей для полного контроля всего:
59      * цвета, кнопки, текст.
60      */
61     private void Awake()
62     {
63         buttonImage = upgradeButton.GetComponent<Image>();
64         defaultButtonColor = buttonImage.color;
65         defaultButtonTextColor = cost_txt.color;
66
67         defaultScaleButton = buttonImage.rectTransform.localScale;
68
69         hexNextLevelColor =
70         ColorUtility.ToHtmlStringRGB(nextLevelColor);
71     }
72     private void Start()
73     {
74         WalletManager.Instance.OnCoinsChanged += UpdateButtonUI;
75         WalletManager.Instance.OnCoinsChanged += UpdateButtonText;
76
77         for (int i = 0; i < 5; i++)
78             CreatePanel();
79     }
80
81     public void SetTower(Tower tower)
82     {
83         ResetInfoMenu();
84         selectedTower = tower;
85         UpdateInfo();
86     }
87
88     public void UpdateInfo()
89     {
90         currentLevelData =
91         selectedTower.towerDef.levels[selectedTower.currentLevel - 1];
92
93         if (selectedTower.currentLevel <
94             selectedTower.towerDef.levels.Length)
95         {
96             nextLevelData =
97             selectedTower.towerDef.levels[selectedTower.currentLevel];
98         }
99         else
100         {
101             nextLevelData = null;
102         }
103
104         iconTower.sprite = selectedTower.towerDef.icon;
105
106         level_txt.text = selectedTower.currentLevel.ToString();
107
108         radius_txt.text =
109         currentLevelData.attackRadiusText.ToString();
110         unitsCount_txt.text =
111         currentLevelData.unitCapacity.ToString();
112
113         if (nextLevelData == null)

```

```

110     {
111         SetMaxLevelStats();
112     }
113     else
114     {
115         cost_txt.text = nextLevelData.cost.ToString();
116     }
117
118     UpdateButtonUI();
119     UpdateScrollRect();
120 }
121
122 /*
123     Когда Plate при нажатии открывает меню информации башни, он
    прокидывает Tower. Далее мы можем узнать его текущий уровень и
    получить нужные данные из TowerDef.
124     При открытии другой плитки данные обновляются. Мы достаём иконку,
    радиус, уровень, количество юнитов. Было решено хранить отдельно
    nextLevelData следующий уровень. Это позволяет нам сравнивать,
    существует ли следующий уровень, и если нет, то это означает
    достижение максимального уровня, и соответственно - устанавливаем
    специальный стиль панели - другие цвета, надписи MAX и прочее.
125 */
126
127 private bool CanUpgrade()
128 {
129     return nextLevelData != null &&
130         WalletManager.Instance.Coins >= nextLevelData.cost;
131 }
132
133 private void UpdateButtonUI(int x)
134 {
135     UpdateButtonUI();
136 }
137
138 private void UpdateButtonUI()
139 {
140     if (nextLevelData == null)
141         return;
142
143     bool canUpgrade = CanUpgrade();
144
145     buttonImage.color = canUpgrade
146         ? upgradeButtonColor
147         : defaultButtonColor;
148
149     cost_txt.color = canUpgrade
150         ? upgradeButtonTextcolor
151         : defaultButtonTextcolor;
152
153     if (isHoverButton)
154         SetNextLevelStats();
155 }
156
157 /*
158     При недостатке средств мы наблюдаем кнопки серого цвета, как и
    текст внутри. А вот если монет у нас достаточно, то кнопки
    приобретают оранжевый цвет. Если уровень максимальный - кнопки
    становятся желтым. Вызов подвязан на событие обновления кошелька.
159 */
160
161 private void UpdateButtonText(int x)
162 {
163     UpdateButtonText();
164 }
165
166 private void UpdateButtonText()

```

```

167     {
168         if (!isHoverButton || nextLevelData == null)
169             return;
170
171         cost_txt.text = CanUpgrade() ? "UPGRADE" : "NOT ENOUGH
MONEY";
172     }
173
174     /*
175     В стандартном состоянии, не важно какого цвета кнопка и хватает ли
средств, внутри кнопки отображается стоимость улучшения. При
наведении стоимость меняется на UPGRADE или NOT ENOUGH MONEY в
зависимости от средств.
176     Почему такое решение? Удобно сразу видеть стоимость, нежели
сначала видеть надпись улучшить, а потом при наведении узнавать
стоимость. + Такой паттерн бы не позволял игроку заранее знать
стоимость, без наведения.
177     */
178
179     public void HoverButtonUpgradeTower()
180     {
181         if (nextLevelData == null)
182             return;
183
184         isHoverButton = true;
185         UpdateButtonText();
186
187         if (CanUpgrade())
188             SetNextLevelStats();
189
190         buttonImage.rectTransform.DOKill();
191         buttonImage.rectTransform
192             .DOScale(defaultScaleButton * 0.95f, 0.1f)
193             .SetEase(Ease.OutQuad);
194
195         OnHoverUpgradeTower?.Invoke(nextLevelData.radiusScale);
196     }
197
198     /*
199     Прокидываем при наведении на улучшение наружу радиус атаки
следующего уровня, чтобы оранжевый круг принял нужный размер и игрок
сразу видел разницу улучшения.
200     */
201
202     public void HoverExitButtonUpgrade()
203     {
204         if (selectedTower == null)
205             return;
206
207         ResetButtonScaleAnim();
208
209         OnExitUpgradeTower?.Invoke();
210
211         if (nextLevelData == null)
212             return;
213
214         SetCurrentLevelStats();
215         cost_txt.text = nextLevelData.cost.ToString();
216     }
217
218     public void ClickButton()
219     {
220         if (!CanUpgrade())
221         {
222             OnClickCantUpgrade?.Invoke();
223             return;
224

```

```

225     }
226
227
228     if (selectedTower.TryUpgradeTower())
229     {
230         UpdateInfo();
231         if (nextLevelData != null)
232             ClickUpgradeButton?.Invoke(nextLevelData);
233         else
234             ClickUpgradeButton?.Invoke(null);
235     }
236
237     }
238
239     /*
240     Если мы всё таки смогли улучшить башню и получили True, то мы
241     просто вызываем обновление данных. Сами мы ничего не делаем. Метод
242     TryUpgradeTower уже улучшил у Tower свой уровень. UpdateInfo просто
243     вновь переспросит какой у него уровень.
244     */
245
246     private void UpdateScrollRect()
247     {
248         while (unitPanels.Count <
249             selectedTower.unitLevelsBySlot.Count)
250         {
251             CreatePanel();
252         }
253         for (int i = 0; i != selectedTower.unitLevelsBySlot.Count;
254             i++)
255         {
256             unitPanels[i].gameObject.SetActive(true);
257             unitPanels[i].SetUnit(selectedTower.unitDef,
258                 selectedTower.unitLevelsBySlot[i], i);
259         }
260     }
261
262     /*
263     CreatePanel вызывается на старте и создаёт внутренний пул из 5
264     панелей. Позже мы перепроверяем, хватает ли их, и просто заполняем
265     каждую панель нужным юнитом.
266     */
267
268     private UnitPanel CreatePanel()
269     {
270         {
271             var panel = Instantiate(prefabPanelUnits, contentRoot)
272                 .GetComponent<UnitPanel>();
273
274             panel.gameObject.SetActive(false);
275
276             panel.UnitPanelOnClickBuy += BuyUnit;
277             panel.UnitPanelOnClickUp += UpUnit;
278
279             panel.UnitPanelOnClickCantBuy += CantBuyUnit;
280             panel.UnitPanelOnClickCantUp += CantUpUnit;
281             panel.UnitPanelHoverEnter += HoverUnit;
282
283             unitPanels.Add(panel);
284
285             return panel;
286         }
287     }
288
289     /*
290     Создаём объект по префабу и выполняем все подписки.
291     */
292
293     private void BuyUnit(int index)

```

```

284     {
285         if (selectedTower.TryUnitBuy(index))
286             unitPanels[index].SetUnit(
287                 selectedTower.unitDef,
288                 selectedTower.unitLevelsBySlot[index],
289                 index);
290     }
291
292     private void UpUnit(int index)
293     {
294         if (selectedTower.TryUnitUpgrade(index))
295             unitPanels[index].SetUnit(
296                 selectedTower.unitDef,
297                 selectedTower.unitLevelsBySlot[index],
298                 index);
299     }
300
301     /*
302     При нажатии на кнопку панели она передаёт в событие индекс, чтобы
303     панель информации точно знала какого юнита сейчас улучшает игрок.
304     */
305
306     private void CantBuyUnit()
307     {
308         OnCantBuyUnit?.Invoke();
309     }
310
311     private void CantUpUnit()
312     {
313         OnCantUpUnit?.Invoke();
314     }
315
316     private void HoverUnit()
317     {
318         OnHoverUnit?.Invoke();
319     }
320
321
322     private void ResetInfoMenu()
323     {
324         foreach (var unit in unitPanels)
325         {
326             unit.ResetPanel();
327             unit.gameObject.SetActive(false);
328         }
329
330         iconTower.sprite = null;
331         selectedTower = null;
332         level_txt.text = "";
333
334         nextLevelData = null;
335         radius_txt.text = "";
336         unitsCount_txt.text = "";
337
338         cost_txt.text = "";
339     }
340
341     private void SetCurrentLevelStats()
342     {
343         level_txt.text = selectedTower.currentLevel.ToString();
344         radius_txt.text =
345         currentLevelData.attackRadiusText.ToString();
346         unitsCount_txt.text =
347         currentLevelData.unitCapacity.ToString();
348
349         level_txt.color = defaultTextColor;

```

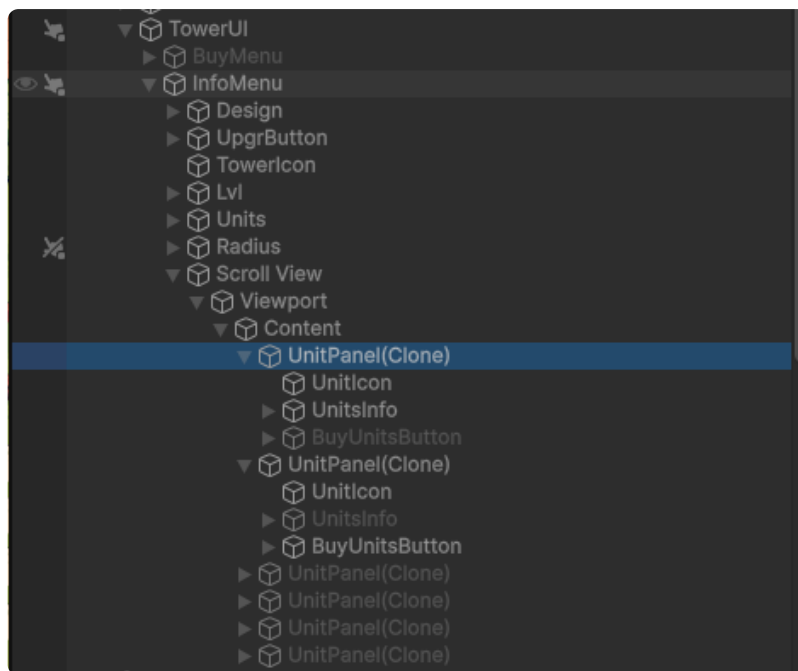
```

348         radius_txt.color = defaultTextColor;
349         unitsCount_txt.color = defaultTextColor;
350     }
351
352     private void SetNextLevelStats()
353     {
354         if (nextLevelData == null)
355             return;
356         level_txt.text =
357             $"{selectedTower.currentLevel} <color=#
{hexNextLevelColor}>> {selectedTower.currentLevel + 1}</color>";
358         radius_txt.text =
359             $"{currentLevelData.attackRadiusText} <color=#
{hexNextLevelColor}>> {nextLevelData.attackRadiusText}</color>";
360         unitsCount_txt.text =
361             $"{currentLevelData.unitCapacity} <color=#
{hexNextLevelColor}>> {nextLevelData.unitCapacity}</color>";
362     }
363
364     private void SetMaxLevelStats()
365     {
366         ResetButtonScaleAnim();
367
368         SetCurrentLevelStats();
369
370         level_txt.text = "MAX";
371         cost_txt.text = "MAX";
372
373         level_txt.color = maxLevelColorText;
374         radius_txt.color = maxLevelColorText;
375         unitsCount_txt.color = maxLevelColorText;
376
377         buttonImage.color = maxButtonColor;
378         cost_txt.color = maxButtonTextColor;
379     }
380
381     private void ResetButtonScaleAnim()
382     {
383         if (isHoverButton)
384         {
385             isHoverButton = false;
386
387             buttonImage.rectTransform.DOKill();
388             buttonImage.rectTransform
389                 .DOScale(defaultScaleButton, 0.1f)
390                 .SetEase(Ease.OutBack);
391         }
392     }
393 }

```



Unit Panels.



Иерархия InfoMenu и панелей юнитов.

```

1 using DG.Tweening;
2 using System;
3 using TMPPro;
4 using UnityEngine;
5 using UnityEngine.UI;
6
7 /*
8  * Теперь нужен скрипт для каждой панели юнитов. По размеру он вышел
9  * как и InfoMenu.
10 */

```

```

10
11 public class UnitPanel : MonoBehaviour
12 {
13     [Header("Main Fields")]
14     public Image iconUnit;
15     public TextMeshProUGUI lvl_txt;
16     public TextMeshProUGUI speed_txt;
17     public TextMeshProUGUI damage_txt;
18
19     [Header("ButtonBuy")]
20     public Button buyUnitButton;
21     public TextMeshProUGUI buyUnitButton_txt;
22     private Image buyUnitBI;
23
24     [Header("ButtonUpgrade")]
25     public Button upgradeUnitButton;
26     public TextMeshProUGUI upgradeUnitButton_txt;
27     private Image upgradeUnitBI;
28
29     [Header("Colors Button")]
30     public Color buttonColorActive;
31     public Color buttonTextColorActive;
32     public Color maxLevelButtonColor;
33
34     [Header("Colors Text")]
35     public Color nextLevelColor;
36     public Color currentLevelColor;
37     public Color maxLevelColor;
38     public Color maxLevelButtonTextColor;
39     private string hexNextLevelColor;
40
41     [Header("Other")]
42     public GameObject unitsInfo;
43
44     private Color defaultColorButton;
45     private Color defaultColorTxtB;
46
47     private Vector3 buyButtonScale;
48     private Vector3 upButtonScale;
49
50     private UnitDefinition unitDef;
51     private UnitLevelData currentLevelData;
52     private UnitLevelData nextLevelData;
53     private int unitLevel;
54
55     private bool isHoverUp = false;
56     private bool isHoverBuy = false;
57
58     private int panelIndex;
59     public event Action<int> UnitPanelOnClickBuy;
60     public event Action<int> UnitPanelOnClickUp;
61
62     public event Action UnitPanelOnClickCantBuy;
63     public event Action UnitPanelOnClickCantUp;
64     public event Action UnitPanelHoverEnter;
65
66     /*
67     35 полей...
68     */
69
70     private void Awake()
71     {
72         buyUnitBI = buyUnitButton.GetComponent<Image>();
73         upgradeUnitBI = upgradeUnitButton.GetComponent<Image>();
74
75         defaultColorButton = buyUnitBI.color;
76         defaultColorTxtB = buyUnitButton_txt.color;

```

```

77
78     buyButtonScale = buyUnitBI.rectTransform.localScale;
79     upButtonScale = upgradeUnitBI.rectTransform.localScale;
80
81     hexNextLevelColor =
ColorUtility.ToHtmlStringRGB(nextLevelColor);
82 }
83 private void Start()
84 {
85     WalletManager.Instance.OnCoinsChanged += UpdateBuyButton;
86     WalletManager.Instance.OnCoinsChanged += UpdateUpButton;
87 }
88
89 public void SetUnit(UnitDefinition unitDef, int level, int
indexPanel)
90 {
91     ResetHoverState();
92
93     this.unitDef = unitDef;
94     unitLevel = level;
95     panelIndex = indexPanel;
96     UpdateUnit();
97 }
98
99 private void UpdateLevelData()
100 {
101     bool hasUnit = unitLevel > 0;
102
103     currentLevelData = hasUnit
104         ? unitDef.unitLevelData[unitLevel - 1]
105         : null;
106
107     nextLevelData = unitLevel < unitDef.unitLevelData.Length
108         ? unitDef.unitLevelData[unitLevel]
109         : null;
110
111     iconUnit.sprite = hasUnit
112         ? currentLevelData.icon
113         : nextLevelData.icon;
114 }
115
116 private void UpdateUnit()
117 {
118     UpdateLevelData();
119
120     if (unitLevel == 0)
121     {
122
123         unitsInfo.SetActive(false);
124
125         UpdateBuyButton();
126         buyUnitButton_txt.text = nextLevelData.cost.ToString();
127         return;
128     }
129
130     buyUnitButton.gameObject.SetActive(false);
131     unitsInfo.SetActive(true);
132
133     if (nextLevelData == null)
134     {
135         SetMaxLevelStats();
136         return;
137     }
138
139     SetCurrentLevelStats();
140     upgradeUnitButton_txt.text = nextLevelData.cost.ToString();
141

```

```

142         UpdateUpButton();
143
144     }
145
146     /*
147     Если уровень юнита нулевой, значит он ещё не куплен и мы скрываем
    блок unitsInfo. UnitPanel состоит из иконки, которая отображается
    всегда, кнопки покупки и блока информации, который скрывается, если
    юнит не куплен.
148     Если юнит присутствует, то мы скрываем кнопку покупки и включаем
    блок информации, где также есть кнопка улучшения.
149     */
150
151     private void UpdateBuyButton(int x)
152     {
153         UpdateBuyButton();
154     }
155     private void UpdateBuyButton()
156     {
157         if (unitLevel > 0 || nextLevelData == null)
158             return;
159
160         bool canBuy = CanBuy();
161
162         buyUnitBI.color = canBuy
163             ? buttonColorActive
164             : defaultColorButton;
165
166         buyUnitButton_txt.color = canBuy
167             ? buttonTextColorActive
168             : defaultColorTxtB;
169
170         if (isHoverBuy)
171         {
172             buyUnitButton_txt.text = canBuy
173                 ? "BUY"
174                 : "NOT ENOUGH MONEY";
175         }
176         else
177         {
178             buyUnitButton_txt.text = nextLevelData.cost.ToString();
179         }
180     }
181
182     /*
183     Также сохраняем логику функционала кнопок. Если монет недостаточно
    - серые, если достаточно - оранжевые. Без наведения - цена, с
    наведением BUY/N.E.M.
184     Обновляем, подписавшись на событие кошелька, или при наведении на
    кнопку.
185     */
186
187     private void UpdateUpButton(int x)
188     {
189         UpdateUpButton();
190     }
191     private void UpdateUpButton()
192     {
193         if (unitLevel == 0 || nextLevelData == null)
194             return;
195
196         bool canUp = CanUpgrade();
197
198         upgradeUnitBI.color = canUp
199             ? buttonColorActive
200             : defaultColorButton;
201

```

```

202         upgradeUnitButton_txt.color = canUp
203             ? buttonTextColorActive
204             : defaultColorTxtB;
205
206         if (isHoverUp)
207             SetNextLevelStats();
208     }
209
210     /*
211     Тоже самое, но для кнопки из блока информации - UP.
212     */
213
214     private bool CanBuy()
215     {
216         return unitLevel == 0 &&
217             nextLevelData != null &&
218             WalletManager.Instance.Coins >= nextLevelData.cost;
219     }
220
221     private bool CanUpgrade()
222     {
223         return nextLevelData != null &&
224             WalletManager.Instance.Coins >= nextLevelData.cost;
225     }
226
227
228
229     public void HoverButtonBuy()
230     {
231         UnitPanelHoverEnter?.Invoke();
232
233         if (unitLevel > 0)
234             return;
235
236         isHoverBuy = true;
237
238         buyUnitBI.rectTransform.DOKill();
239         buyUnitBI.rectTransform
240             .DOScale(buyButtonScale * 0.95f, 0.1f)
241             .SetEase(Ease.OutQuad);
242
243         buyUnitButton_txt.text = CanBuy()
244             ? "BUY"
245             : "NOT ENOUGH MONEY";
246
247     }
248
249     public void HoverExitButtonBuy()
250     {
251         if (unitLevel > 0)
252             return;
253
254         buyUnitBI.rectTransform.DOKill();
255         buyUnitBI.rectTransform
256             .DOScale(buyButtonScale, 0.4f)
257             .SetEase(Ease.OutBack);
258
259         buyUnitButton_txt.text = nextLevelData.cost.ToString();
260
261         isHoverBuy = false;
262     }
263
264     public void ClickButtonBuy()
265     {
266         if (unitLevel > 0 || !CanBuy())
267         {
268             UnitPanelOnClickCantBuy?.Invoke();

```

```

269         return;
270     }
271
272     UnitPanelOnClickBuy?.Invoke(panelIndex);
273 }
274
275 /*
276     Пробрасываем индекс панели в событие, чтобы InfoMenu знал, какой
277     юнит был куплен.
278 */
279
280 public void HoverButtonUp()
281 {
282     UnitPanelHoverEnter?.Invoke();
283
284     if (nextLevelData == null)
285         return;
286
287     upgradeUnitBI.rectTransform.DOKill();
288     upgradeUnitBI.rectTransform
289         .DOScale(upButtonScale * 0.95f, 0.1f)
290         .SetEase(Ease.OutBack);
291
292     isHoverUp = true;
293
294     bool canUpgrade = CanUpgrade();
295
296     upgradeUnitButton_txt.text = canUpgrade
297         ? "UPGR"
298         : "NOT ENOUGH MONEY";
299
300     if (canUpgrade)
301         SetNextLevelStats();
302 }
303 public void HoverExitButtonUp()
304 {
305     if (nextLevelData == null)
306         return;
307
308     ResetUpButtonScaleAnim();
309
310     SetCurrentLevelStats();
311
312     upgradeUnitButton_txt.text = nextLevelData.cost.ToString();
313
314     isHoverUp = false;
315 }
316 public void ClickButtonUp()
317 {
318     if (nextLevelData == null || !CanUpgrade())
319     {
320         UnitPanelOnClickCantUp?.Invoke();
321         return;
322     }
323
324     UnitPanelOnClickUp?.Invoke(panelIndex);
325 }
326
327 private void SetCurrentLevelStats()
328 {
329     lvl_txt.text = unitLevel.ToString();
330     speed_txt.text = $"{currentLevelData.attackCooldown}s";
331     damage_txt.text = currentLevelData.damage.ToString();
332
333     lvl_txt.color = currentLevelColor;
334     speed_txt.color = currentLevelColor;

```

```

335         damage_txt.color = currentLevelColor;
336     }
337
338     /*
339     Также устанавливаем значения текущего уровня.
340     */
341
342     private void SetNextLevelStats()
343     {
344         if (nextLevelData == null || currentLevelData == null)
345             return;
346
347         lvl_txt.text =
348             $"{unitLevel} <color=#{hexNextLevelColor}>>
{nextLevelData.level}</color>";
349         speed_txt.text =
350             $"{currentLevelData.attackCooldown}s <color=#{
{hexNextLevelColor}>> {nextLevelData.attackCooldown}s</color>";
351         damage_txt.text =
352             $"{currentLevelData.damage} <color=#{hexNextLevelColor}>>
{nextLevelData.damage}</color>";
353     }
354
355     /*
356     При наведении мы также вызываем установку следующих значений,
    чтобы видеть изменения в цифрах. Формат: белый текст текущего уровня
    -> оранжевый текст следующего уровня.
357     */
358
359     private void SetMaxLevelStats()
360     {
361         SetUpButtonScaleAnim();
362         SetCurrentLevelStats();
363
364         lvl_txt.text = "MAX";
365         upgradeUnitButton_txt.text = "MAX";
366
367         lvl_txt.color = maxLevelColor;
368         speed_txt.color = maxLevelColor;
369         damage_txt.color = maxLevelColor;
370
371         upgradeUnitBI.color = maxLevelButtonColor;
372         upgradeUnitButton_txt.color = maxLevelButtonTextcolor;
373
374     }
375
376     private void ResetUpButtonScaleAnim()
377     {
378         if (!isHoverUp)
379             return;
380         upgradeUnitBI.rectTransform
381             .DOScale(upButtonScale, 0.6f)
382             .SetEase(Ease.OutBack);
383     }
384
385     public void ResetPanel()
386     {
387         unitsInfo.SetActive(false);
388         buyUnitButton.gameObject.SetActive(true);
389
390         iconUnit.sprite = null;
391
392         lvl_txt.text = "";
393         speed_txt.text = "";
394         damage_txt.text = "";
395
396         unitDef = null;

```

```

397         unitLevel = 0;
398
399         currentLevelData = null;
400         nextLevelData = null;
401     }
402
403     /*
404     Т.к. панели мы переиспользуем, а не пересоздаём, как тот же визуал
405     башен, то не забываем сбрасывать данные.
406     */
407     private void ResetHoverState()
408     {
409         isHoverBuy = false;
410         isHoverUp = false;
411
412         buyUnitButton_txt.text = "";
413         upgradeUnitButton_txt.text = "";
414     }
415 }

```

- ✓ Полная система заняла достаточно не мало времени. Она получилась расширяемой, относительно чистой и простой. Значимую часть кода занимают примитивные штуки по типу присвоения цвета кнопки в некоторых условиях, или смена текста надписей характеристик. Самое сложное было, пожалуй, вместить панель в границы экрана и не дать ей выйти за них.

Изменения скорее всего неизбежны, т.к. планируется создание различных предметов улучшений, которые повлияют на эту логику. Скорее всего, больше внедрений ещё не созданного GameModifiers, который сможет менять как простые параметры, так и некоторую часть логики.

Созданные скрипты: TowerUIManager, InfoMenu, UnitPanel, BuyMenu, TowerBuyPlate, Plate, Tower, TowerRange, TowerVisual, Unit, UnitAnimationController, UnitAttack, Projectile, Arrow, IDamageable.

